

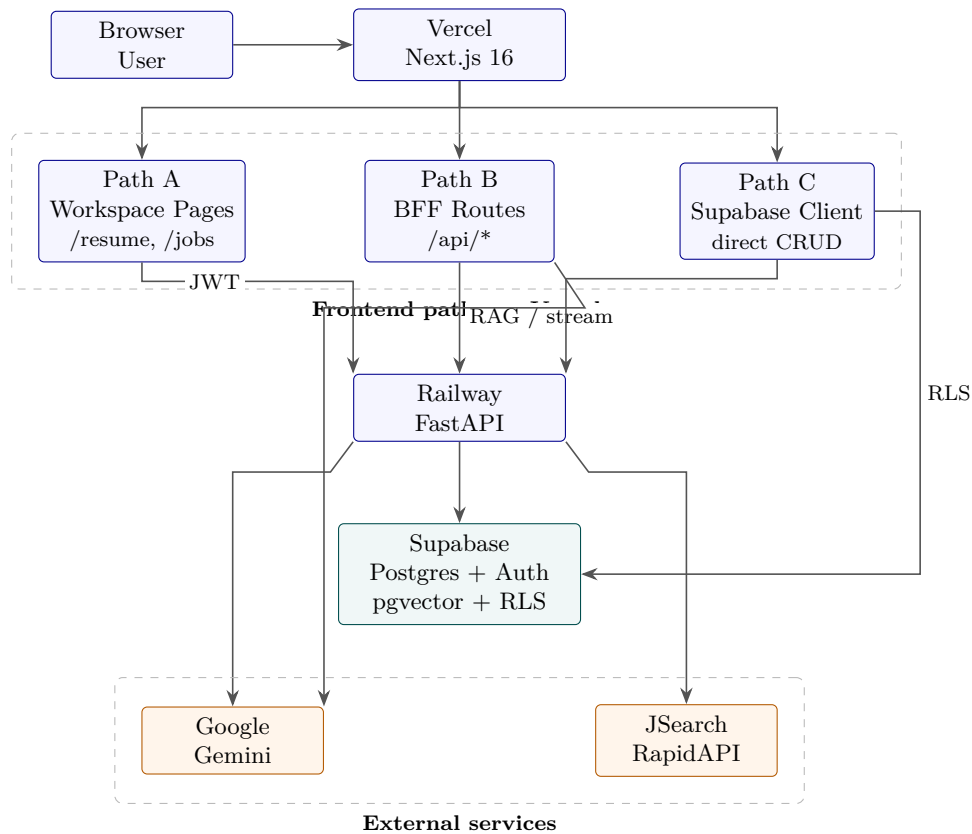
CareerPilot — System Design

Next.js 16 • FastAPI • Supabase (pgvector) • Gemini • JSearch

1. Data Flow Diagrams

Note: These are *data-flow* views (where data moves and transforms). Four figures, one per page, top-to-bottom reading order left-to-right within each figure.

Figure 1 — System-Level Data Flow



Path A (FastAPI): CV ingest, job search, fit scoring, tracker — Bearer JWT to Railway.

Path B (BFF): LLM streaming for chat, cover letters, roadmaps, dashboard nudges.

Path C (Direct): Calendar events and standalone tasks — Supabase anon key + RLS.

Figure 2 — CV Upload & RAG Core (Single Source of Truth)

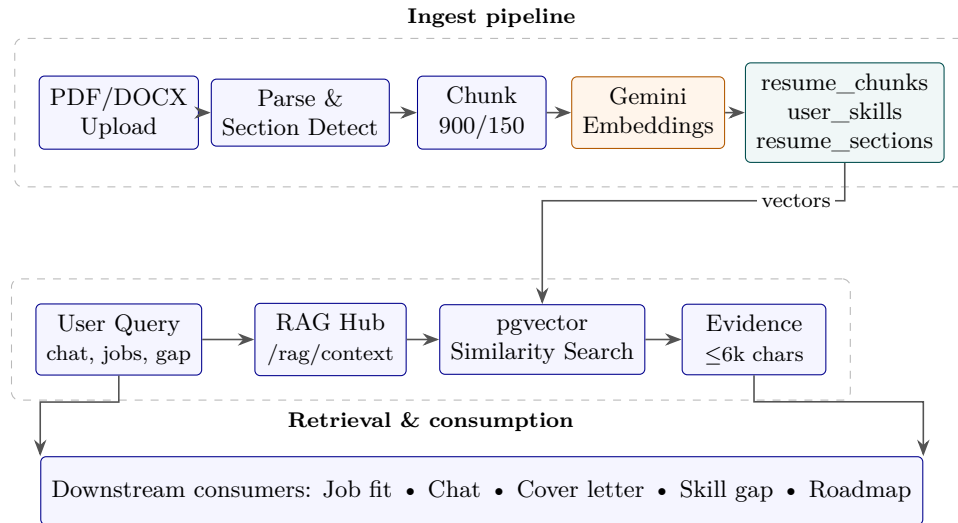


Figure 3 — Job Search & Fit Scoring Data Flow

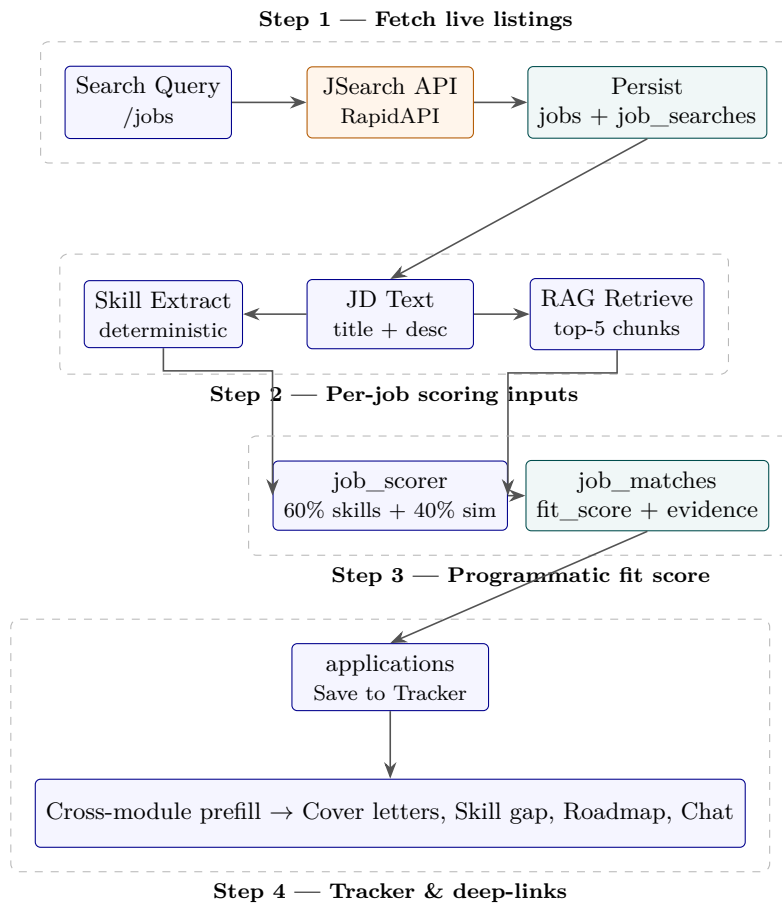
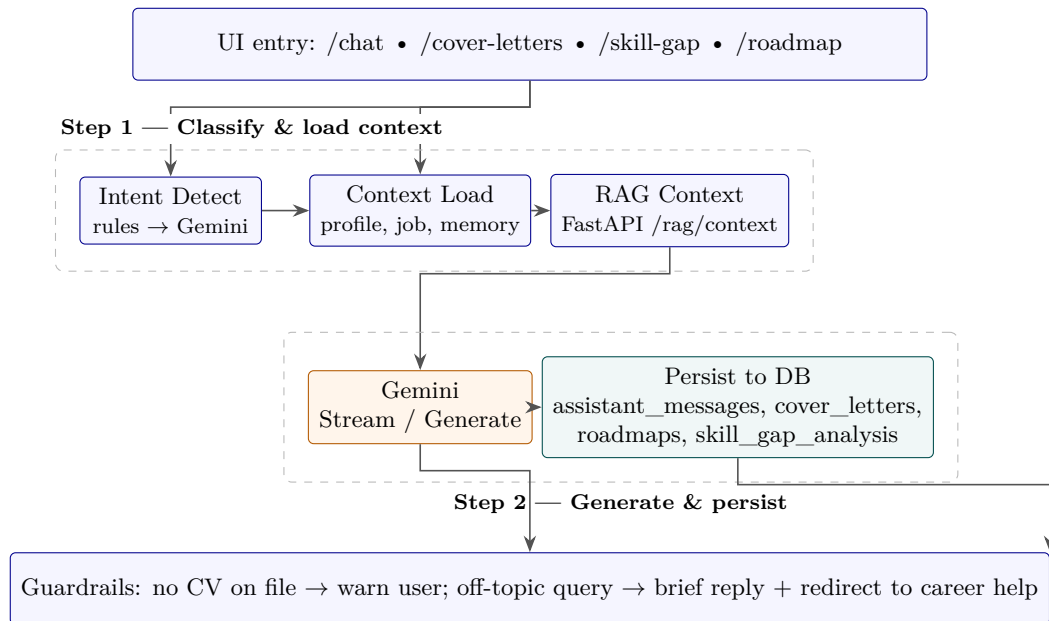


Figure 4 — Assistant Chat & Career Artifact Generation



2. Scaling Analysis (10,000 Users)

Planning assumption: 10,000 monthly active users (MAU), moderate usage (1 CV upload, 3 job searches, 5 chat turns, 2 AI workflows per user/month).

Current architecture supports approximately 500 concurrent users.

At peak, a single Railway FastAPI instance, Vercel serverless concurrency limits, synchronous CV processing, and Supabase connection caps bound throughput before horizontal scaling, async workers, and shared caching are applied.

To support 10,000 users:

1. **Async CV processing** — Move parse → chunk → embed off the upload request path; enqueue to Upstash Redis; Railway worker updates `resumes.status`; frontend polls or uses realtime.
2. **Horizontal API scaling** — Run 3–4 stateless FastAPI instances on Railway behind a load balancer; JWT carries identity (no sticky sessions).
3. **Database tier upgrade** — Supabase Pro with connection pooler (transaction mode for serverless, session mode for workers); covering indexes on `applications(user_id)`, `job_matches(user_id)`, `resume_chunks(user_id)`.
4. **Centralize AI calls** — Route all Gemini traffic through FastAPI (or a dedicated AI gateway); enforce per-user daily quotas; default to Flash-class models.
5. **Shared cache layer** — Replace in-memory nudge Map with Upstash Redis (TTL 15–60 min); cache JSearch (`query`, `location`) responses.
6. **Vector index evolution** — Keep IVFFlat for <50k chunks; migrate to HNSW when `resume_chunks` exceeds ~500k rows; align embedding dimension across config, column, and RPC.
7. **Job deduplication** — Unique constraint on (`source`, `source_url`) with upsert to prevent storage bloat from repeat searches.
8. **Vercel Pro tuning** — Extended serverless timeouts (60s+) for chat/nudge BFF routes; React Query `staleTime` to reduce dashboard re-fetches.
9. **Observability** — Request IDs across BFF → FastAPI; metrics for p95 RAG latency, Gemini token usage, JSearch 429 rate.
10. **Scheduled reminders** — Add cron / workflow service (Supabase Edge Functions, Inngest) for calendar `reminder_time` delivery.

Traffic at 10k MAU (planning): ~10k CV uploads • ~30k JSearch calls • ~50k chat turns • ~20k generation workflows • ~200k lightweight DB reads.

3. Estimated Cost per User / Month

Planning estimates from public pricing (June 2026). Not billing guarantees. Moderate AI usage profile at 10,000 MAU.

Component	Planning choice	Est. USD/mo
Backend API	Railway (2 web services + 1 worker)	\$45–\$80
Queue	Upstash Redis (pay-as-you-go)	\$10
Database / Auth / Vector	Supabase Pro (+ modest compute)	\$75
Job search	JSearch Pro / Ultra (30k searches/mo)	\$30–\$75
Gemini AI	Embeddings + generation (moderate usage)	\$25–\$120
Frontend (ref.)	Vercel Pro	\$20
Observability	Logs, overages, storage buffer	\$15–\$25
Total		\$220–\$405

Cost per active user per month:

$$\begin{aligned}
 C_{\text{user}} &= \frac{C_{\text{fixed}} + C_{\text{Gemini}} + C_{\text{JSearch_overage}}}{N_{\text{MAU}}} \\
 &\approx \frac{\$220\text{--}\$405}{10,000} \\
 &\approx \$0.022\text{--}\$0.041 \text{ / user / month}
 \end{aligned}$$

Rounded planning figure: **~\$0.03 per MAU per month** at moderate usage.

Variable Cost Drivers (Formulas)

JSearch:

$$\begin{aligned}
 R &= N_{\text{MAU}} \times s_{\text{user}} \\
 (\text{e.g. } R &= 10,000 \times 3 = 30,000 \text{ req/mo}) \\
 C_{\text{JSearch}} &= P_{\text{plan}} + \max(0, R - R_{\text{included}}) \times r_{\text{overage}}
 \end{aligned}$$

Gemini embeddings:

$$C_{\text{embed}} = \frac{T_{\text{CV}} + T_{\text{query}}}{10^6} \times p_{\text{embed}}$$

Gemini generation:

$$\begin{aligned}
 C_{\text{gen}} &= \frac{T_{\text{in}}}{10^6} \times p_{\text{in}} \\
 &\quad + \frac{T_{\text{out}}}{10^6} \times p_{\text{out}}
 \end{aligned}$$

Total variable AI:

$$C_{\text{Gemini}} = C_{\text{embed}} + C_{\text{gen}}$$

where T = tokens, p = per-million rates from Gemini pricing. At 10 AI workflows/user/month, C_{Gemini} can exceed \$120 and dominate total cost.

Per-user total:

$$C_{\text{user}} = \frac{C_{\text{fixed}} + C_{\text{Gemini}} + C_{\text{JSearch}}}{N_{\text{MAU}}}$$

4. Key Bottlenecks and Mitigations

#	Bottleneck	Risk at Scale	Evidence in Code-base	Mitigation
1	AI API Usage (Gemini)	Many features rely on Gemini. As user numbers grow, API costs and rate limits may become a problem.	Gemini is used for chat, resume analysis, cover letters, interview preparation, roadmaps, and skill-gap analysis.	Add daily usage limits per user, cache repeated requests, and use lower-cost Gemini models where possible.
2	Resume Processing During Upload	Upload requests take longer because resume parsing, embedding generation, and database updates happen immediately. Multiple uploads at once could slow the system.	Resume processing runs directly during the upload request.	Move processing to a background worker and notify users when processing is complete.
3	Job Search API Dependency	If the external Job Search API becomes slow or reaches its quota limit, job recommendations may stop working.	All job searches depend on the JSearch API.	Cache search results, reduce duplicate searches, and upgrade the API plan if needed.
4	Multiple AI Entry Points	AI requests are handled from several different places, making monitoring and rate limiting difficult.	Both FastAPI and Next.js routes communicate with Gemini.	Route all AI requests through a single back-end service.
5	In-Memory Cache	Cached data is stored only on one server instance. In a multi-server deployment, users may receive inconsistent results.	Nudge cache currently uses an in-memory Map.	Replace with Redis or another shared cache system.
6	Vector Search Growth	As more resumes are stored, similarity search may become slower and less accurate.	Resume embeddings are stored using pgvector.	Upgrade indexing strategy and archive old data when necessary.
7	Database Connection Limits	Many simultaneous users may exceed the database's connection limit.	Multiple services create database connections independently.	Use connection pooling and limit unnecessary database requests.
8	Duplicate Job Listings	Repeated job searches can store the same job multiple times, increasing storage usage.	Job records are inserted without strong duplicate checking.	Add unique constraints and use upsert operations.
9	Reminder Delivery	Reminder times are stored, but notifications are not automatically sent to users.	Reminder data exists but no scheduler is configured.	Add scheduled jobs using cron or a workflow service.
10	Limited Monitoring	Production issues may be difficult to diagnose due to limited monitoring and metrics collection.	Basic logging exists, but no centralized monitoring.	Add metrics, request tracing, and performance dashboards.
11	File Storage Reliability	If a resume file upload fails silently, the resume cannot be processed later.	Storage upload errors are not always enforced.	Verify file uploads before marking resumes as successfully processed.

#	Bottleneck	Risk at Scale	Evidence in Code-base	Mitigation
12	Job Matching Computation	Every job search performs scoring calculations for many jobs, increasing CPU usage.	Job matching is performed for each returned job.	Limit the number of jobs scored and precompute frequently used embeddings.

5. Highest-Priority Fixes Before Production

1. Move resume processing to an asynchronous background worker.
2. Centralize all Gemini API calls and add per-user usage limits.
3. Replace the in-memory cache with Redis.
4. Verify embedding/vector configuration consistency.
5. Add caching for external job search API responses.

CareerPilot System Design — derived from Docs/submit/system-design.md.

Pricing refs: <https://railway.app/pricing> • <https://supabase.com/pricing> • <https://ai.google.dev/pricing> • <https://upstash.com/pricing> • JSearch via RapidAPI.